MGG Arena Game Integration Guide (Full Version)

> A **full-version** document containing **structure, implementation guide, and full API specifications** for game developers integrating with the MGG Arena platform.
> Includes request/response JSON examples, field tables, and error codes.

Document structure

---

1. Overview

- **Purpose**: Defines APIs used when the game server requests entry fee deduction, result submission, reward distribution, balance/user verification, etc. from the MGG Arena server, and APIs the game server must provide when MGG Arena Admin sends room, reward, and item information to the game server.
- **Game client**: Users log in on the MGG Arena web app and enter via **game URL + game_token**.
- **Game server**: Validates the token and then calls the MGG Arena API with an **API Key**, and implements the APIs called by Admin.

---

2. Integration Structure Overview

2.1 Roles

Role	Owner	Description
MGG Arena Platform	MGG Arena	Login, eMGG balance, room/reward/fee settings, result processing and reward distribution
Game Client	Game developer	Login on MGG Arena web, then enter via game URL
Game Server	Game developer	Matchmaking, game flow, result collection, then calls MGG Arena APIs

2.2 Overall Flow

```
[User] Log in on MGG Arena web
    → Click "Play"
```

```
    → Platform creates game_token and redirects to game URL
    → e.g. https://your-game.com/play?game_token=xxx

[Game Client] Receives game_token + (optional) access_token, refresh_token
    → Sends to game server

[Game Server]
    ① User verification: auth/login (verify user/balance with access_token
etc.)
    ② On match: games/match/start (check entry eligibility only)
    ③ On game start: games/start (deduct entry fees for all)
    ④ On game end: games/result (submit rankings → MGG Arena distributes
rewards)
```

---

3. Prerequisites (MGG Arena Side)

The following must be configured by MGG Arena before integration.

3.1 Game and API Key

- Game is registered and an **API Key** for that game is issued.
- All API requests must include `Authorization: Bearer {api_key}`.
- Store the API Key **only on the game server**; do not expose it to the client.

3.2 Room Settings

- **game_rooms**: `room_id`, `condition_amount` (minimum balance), etc. are registered.
- The game server may only use these `room_id` values; other games' `room_id`s are not valid.
- `room_id`: Room identifier used in the game (e.g. `"beginner"`, `"ranked_1000"`).

3.3 Reward Rates

- **room_reward_rates**: Rank-based **reward rates** per (room_id, player_count) are set.
- Example: 4-player room — 1st 50%, 2nd 30%, 3rd 20%.
- The game result API sends only `results` (rankings); MGG Arena calculates and distributes rewards from these rates.

3.4 When Using Bots

- Games using bots must have the bot nickname (`bot_id`) registered in **game_accounts** for that game.
- On **game start**, entry fees are deducted from the bot account(s); ensure sufficient eMGG for bots.
- Bot nicknames may be duplicated in `nicknames` / `results` (e.g. two `"Bot"` entries).

3.5 When Using Items

- For in-game items sold for eMGG, **game_items** holds `item_id`, price, etc.
- The **items/purchase** API may only use `item_id`s registered in that table.

---

4. Game Server Implementation Guide

4.1 User Entry and Verification

1. **Entry URL**
   MGG Arena redirects to `https://your-game-url?game_token=xxx`.
   Delivery of `access_token`, `refresh_token`, etc. can be agreed separately if needed.

2. **game_token verification**
   game_token is a one-time (or time-limited) token issued by MGG Arena (e.g. create-game-token).
   Map it to your session/user; for API calls use **uid + nickname + (access_token, refresh_token)**.

3. **auth/login**
   Use when the game server needs to verify user identity and balance.
   Request body: `uid`, `nickname`, `access_token`, `refresh_token`.
   Response includes `emgg_balance`, `is_verified`, etc.
   On 401 (token expired/invalid), call **auth/refresh** and retry.

4. **Balance only**
   Use **users/balance** with `uid` and `nickname` to query eMGG balance only.

4.2 Single-Game Flow (Required Order)

Follow this order strictly.

1. **Match start** `POST /api/v1/games/match/start`
   When the user attempts to enter a room, **only check eligibility**.
   Request: `room_id`, `user_id`, `nickname`, `entry_fee`.
   No balance deduction here.
   On failure: 400 (room not found, insufficient balance, entry condition not met, etc.).

2. **Game start** `POST /api/v1/games/start`
   Call **once** when matching is complete and the game actually starts.
   Entry fees are **deducted for all (users + bots)**.
   Request: `room_id`, `room_sequence`, `entry_fee`, `player_count`, `bot_count`, `user_count`, `nicknames`, `game_start_time`.
   `room_sequence` is the round number for that room; use the same value when submitting the result.
   On failure: 400 (room/fee not set, bot account missing/insufficient balance, player validation failed, etc.).

3. **Game result** `POST /api/v1/games/result`
   Call **once** when the game ends.
   Request: `room_id`, `room_sequence`, `entry_fee`, `player_count`, `bot_count`, `user_count`, `results` (nickname + rank), `game_start_time`, `game_end_time`.
   Rewards are calculated and distributed by MGG Arena from room/player-count reward rates.
   Duplicate submit for the same `room_id` + `room_sequence` returns 400.

4.3 room_sequence Management

- `room_sequence` must be **unique per game** within the same `room_id`.

- Use an incrementing integer per room (1, 2, 3, …).
- **games/start** and **games/result** must use the same `room_id` + `room_sequence` for correct processing.

4.4 nicknames / results Consistency

- **games/start** `nicknames` and **games/result** `results` must represent the **same set of players**.
- In `results`, send `{ "nickname": "name", "rank": 1 }` in **rank order from 1st**.
- Bots may share the same nickname (e.g. two `"Bot"` entries).

4.5 Timestamps

- Send `game_start_time` and `game_end_time` in **UTC ISO 8601** (e.g. `"2024-01-01T12:00:00.000Z"`).
- The `game_start_time` in **games/result** must match the one sent in **games/start**.

4.6 Error Handling and Retries

- **400:** Missing fields, invalid room_id/item_id, insufficient balance, game already finished, duplicate result. Fix the cause before retrying; do not resend the same request unchanged.
- **401:** Invalid/expired API Key. Check key and retry.
- **500:** May be transient; retry after a short delay. For **games/result**, consider checking whether the request already succeeded before retrying to avoid duplicate submission.

4.7 Item Purchase

- **items/purchase:** Send `user_id`, `item_id`, `item_count`.
- `item_id` must be registered for that game in MGG Arena.
- Fees are applied according to MGG Arena settings.

---

5. Checklist (Before Development Complete)

- ☐ API Key stored only on server (e.g. env); not exposed to client
- ☐ On user entry: receive and verify game_token (and access_token/refresh_token if used)
- ☐ On auth/login failure: retry after refresh
- ☐ Order: match start → game start → game result
- ☐ Use only room_id values registered in MGG Arena
- ☐ Manage room_sequence uniquely per room
- ☐ room_id, room_sequence, player count, nicknames, ranks consistent between games/start and games/result
- ☐ Send game_start_time / game_end_time in UTC ISO 8601
- ☐ Avoid duplicate result submission (especially on retry)
- ☐ When using bots: bot account and balance prepared on MGG Arena side
- ☐ When using items: item_id registered for the game
- ☐ Game server provided API (rooms/rewards/items) implemented and API Key validated (see Section 7)

---

6. MGG Arena Integration API (Game Server → MGG Arena)

Full specification of the REST API used when the game server calls MGG Arena.

6.1 Overview

6.1.1 Purpose

- APIs used when the **game server** requests **entry fee deduction**, **result submission**, **reward distribution**, and **balance/user verification** from the MGG Arena server.
- The game client enters via **game URL + game_token** after logging in on the MGG Arena web app. The game server validates this token and then calls the APIs below using an **API Key**.

6.1.2 Common

Item	Description
Base URL	`https://adminpre.mggarena.com` (dev/staging); production URL provided separately
Authentication	All requests require `Authorization: Bearer {api_key}` (game-specific API Key)
Content-Type	`application/json`
Encoding	UTF-8

6.1.3 API Key

- API Keys are issued after game registration with MGG Arena.
- One API Key per game; **room_id** / **item_id** must be values registered for that game.
- Use the API Key **only on the game server**; do not expose it to clients or logs.

6.2 Authentication and Users

6.2.1 Access Token Verification (Login)

Used when the game server verifies the user token received from the platform.

Item	Value
URL	`POST /api/v1/auth/login`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

- `uid`: MGG Arena platform user ID (UUID format, e.g. `550e8400-e29b-41d4-a716-446655440000`). Must match the user identifier issued at platform login.

```
{
  "uid": "550e8400-e29b-41d4-a716-446655440000",
  "nickname": "Player1",
  "access_token": "access_token from platform",
  "refresh_token": "refresh_token from platform"
}
```

Response (200 OK)

```
{
  "success": true,
  "user": {
    "uid": "550e8400-e29b-41d4-a716-446655440000",
    "nickname": "Player1",
    "emgg_balance": 1000,
    "is_verified": true
  },
  "server_time": "2024-01-01T12:00:00.000Z"
}
```

Error responses

HTTP	Description	Example message
400	Missing fields	Missing required fields
401	Token invalid/expired	Token has been invalidated / Token expired
404	User not found	User not found

6.2.2 Refresh Token

Used to obtain new tokens when the access token has expired.

Item	Value
URL	`POST /api/v1/auth/refresh`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

```
{
  "uid": "550e8400-e29b-41d4-a716-446655440000",
  "refresh_token": "current refresh_token"
}
```

Response (200 OK)

```
{
  "success": true,
  "access_token": "new access_token",
  "refresh_token": "new refresh_token",
  "expires_in": 3600,
  "server_time": "2024-01-01T12:00:00.000Z"
}
```

6.2.3 User eMGG Balance

Item	Value
URL	`POST /api/v1/users/balance`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

```
{
  "uid": "550e8400-e29b-41d4-a716-446655440000",
  "nickname": "Player1"
}
```

Response (200 OK)

```
{
  "success": true,
  "data": {
    "uid": "550e8400-e29b-41d4-a716-446655440000",
    "nickname": "Player1",
    "emgg_balance": 1000
  }
}
```

Errors

- 400: User not found / Invalid nickname

6.3 Game Flow APIs

Call in order: **Match start → Game start → Game result submit**.

6.3.1 Game Match Start

Called when a user attempts to enter a room. **Only checks eligibility**; no balance deduction.

Item	Value
URL	`POST /api/v1/games/match/start`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

```
{
  "room_id": "room ID registered in MGG Arena",
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "nickname": "Player1",
  "entry_fee": 1000
}
```

Response (200 OK)

```
{
  "success": true,
  "current_balance": 10000,
  "server_time": "2024-01-01T12:00:00.000Z"
}
```

Errors

- 400: User profile not found / Room not found or condition_amount/game_id not set
- 400: Insufficient balance for room entry / Insufficient balance for entry fee (message includes required vs available)

Note

- `room_id` must be registered for the game of this API Key.
- Actual balance deduction happens in **Game start**.

6.3.2 Game Start

Called when matching is complete and the game starts. **Entry fees are deducted** for all players (including bots).

Item	Value
URL	`POST /api/v1/games/start`
Auth	`Authorization: Bearer {api_key}`

Item	Value
Content-Type	`application/json`

Request body

```
{
  "room_id": "room_id",
  "room_sequence": 1,
  "entry_fee": 1000,
  "player_count": 4,
  "bot_count": 2,
  "user_count": 2,
  "nicknames": ["Player1", "Player2", "Bot1", "Bot2"],
  "game_start_time": "2024-01-01T12:00:00.000Z"
}
```

Field	Type	Description
room_id	string	Room ID
room_sequence	number	Game round number for this room; use same value when submitting result
entry_fee	number	Entry fee per player (eMGG)
player_count	number	Total players (users + bots)
bot_count	number	Number of bots
user_count	number	Number of real users
nicknames	string[]	Nicknames in player order (duplicate bot nicknames allowed)
game_start_time	string	Game start time (ISO 8601 UTC)

Response (200 OK)

```
{
  "success": true,
  "game_id": 1,
  "room_sequence": 1,
  "total_pot": 3600,
  "player_count": 4,
  "bot_count": 2,
  "user_count": 2,
  "game_start_time": "2024-01-01T12:00:00.000Z",
  "server_time": "2024-01-01T12:00:00.000Z"
}
```

Errors

- 400: Room not found / Game fee rate not found
- 400: Bot profile not found / Insufficient bot balance / Bot account not configured
- 400: Player validation failed (details in validation_errors array)

6.3.3 Game Result Submit

Submit rankings when the game ends. **Rewards are calculated and distributed by the server.**

Item	Value
URL	`POST /api/v1/games/result`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

```
{
  "room_id": "room_id",
  "room_sequence": 1,
  "entry_fee": 1000,
  "player_count": 4,
  "bot_count": 2,
  "user_count": 2,
  "results": [
    { "nickname": "Player1", "rank": 1 },
    { "nickname": "Bot", "rank": 2 },
    { "nickname": "Bot", "rank": 3 },
    { "nickname": "Player2", "rank": 4 }
  ],
  "game_start_time": "2024-01-01T12:00:00.000Z",
  "game_end_time": "2024-01-01T12:30:00.000Z"
}
```

Field	Description
room_id, room_sequence, entry_fee, player_count, bot_count, user_count	Same as game start
results	Array of `{ nickname, rank }`. Ranks must be consecutive integers from 1
game_start_time	Same value as in game start
game_end_time	Game end time (ISO 8601 UTC)

Response (200 OK)

```
{
  "success": true,
```

```
  "result_id": 67890,
  "total_pot": 3600,
  "player_count": 4,
  "rewards_distributed": true,
  "game_end_time": "2024-01-01T12:30:00.000Z",
  "server_time": "2024-01-01T12:30:00.000Z"
}
```

Errors

- 400: Game start not found / Game already finished / Game result already processed
- 400: Player count mismatch / Invalid rank sequence / Duplicate nickname / Player not found in game_start
- 400: Bot profile not found / Reward rates not set for this room/player count

Note

- `reward_amount` is calculated by the server from room/player-count reward rates.
- Duplicate submit for same `room_id` + `room_sequence` is not allowed.

6.4 Items

6.4.1 Item Purchase

Item	Value
URL	`POST /api/v1/items/purchase`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

```
{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "item_id": "item ID registered in MGG Arena",
  "item_count": 5
}
```

Response (200 OK)

```
{
  "success": true,
  "item_price": 50,
  "item_count": 5,
  "total_cost": 250,
  "remaining_balance": 750
}
```

Errors

- 400: Insufficient balance / Item not found or price not set / User profile not found / Item fee rate not found
- `item_id` must be registered for the game of this API Key.

6.5 HTTP Status Codes and Errors

Code	Meaning
200	Success
400	Bad Request (missing required fields, business rule violation, etc.)
401	Unauthorized (API Key missing/expired/invalid)
403	Forbidden
404	Not Found
500	Internal Server Error

Error body example:

```
{
  "success": false,
  "error": "Error message",
  "room_id": "room_001",
  "validation_errors": []
}
```

6.6 Security and Operations

- **API Key**
  - Use only on the game server backend; do not include in client, logs, or URLs.
- **HTTPS**
  - All requests must use HTTPS.
- **Timestamps**
  - Use UTC ISO 8601 for `game_start_time` and `game_end_time`.
- **Retries**
  - Use short backoff and retry for 500/transient errors. Avoid duplicate result submission.

---

7. Game Server Provided API (MGG Arena → Game Server)

Full specification of APIs used when MGG Arena **Admin** calls the game server. The **game server must implement** these APIs and conform to the request/response formats below.

7.1 Overview

7.1.1 Purpose

- When MGG Arena Admin **creates, updates, or deletes Rooms**, **Reward Rates**, or **Items**, it calls the **game server** APIs.
- The game server implements these APIs, validates requests with the **API Key**, applies the request body, and returns a response.

7.1.2 Common

Item	Description
Base URL	`main_url` configured at game API registration (e.g. `https://game.example.com`)
Paths	Relative paths from `game_rooms_url`, `game_reward_rates_url`, `game_items_url` (e.g. `rooms`, `rewards`, `items`)
Authentication	All requests include `Authorization: Bearer {api_key}`. The game server must verify this key.
Content-Type	`application/json`
Encoding	UTF-8

7.1.3 URL Structure

- **Request URL** = `{main_url}/{service_path}` or `{main_url}/{service_path}/{resource_id}`, etc.
- Example: `main_url` = `https://game.example.com`, `game_rooms_url` = `rooms`
  - Create room: `POST https://game.example.com/rooms`
  - Update room: `PUT https://game.example.com/rooms/room_001`
  - Room status: `PATCH https://game.example.com/rooms/room_001/status`

7.2 Rooms API

7.2.1 Create Room

Called when Admin creates a room.

Item	Value
Method	`POST`
URL	`{main_url}/{game_rooms_url}`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

Field	Type	Required	Description

Field	Type	Required	Description
room_id	string	Y	Room identifier (value created in MGG Arena)
reward_type	string	Y	Reward type (e.g. TOP_WINNER, RANKING)
entry_fee	number	Y	Entry fee (eMGG)
min_players	number	Y	Minimum number of players
max_players	number	Y	Maximum number of players
entry_condition	number	Y	Minimum balance to enter room (eMGG)

Request example

```
{
  "room_id": "room_beginner_100",
  "reward_type": "RANKING",
  "entry_fee": 1000,
  "min_players": 2,
  "max_players": 4,
  "entry_condition": 5000
}
```

Response (200 OK)

Return HTTP 200 with a JSON body. Admin does not parse the body; it only checks success.

```
{
  "success": true,
  "room_id": "room_beginner_100"
}
```

Error responses

HTTP	Description
400	Bad Request (missing fields, format error, etc.)
401	Unauthorized (API Key missing/mismatch)
409	Conflict (e.g. room_id already exists)
500	Internal Server Error

Error body example:

```
{
  "success": false,
```

```
  "error": "Room already exists",
  "room_id": "room_beginner_100"
}
```

7.2.2 Update Room

Called when Admin updates room information.

Item	Value
Method	PUT
URL	{main_url}/{game_rooms_url}/{room_id}
Auth	Authorization: Bearer {api_key}
Content-Type	application/json

Request body

Field	Type	Required	Description
room_id	string	Y	Room identifier (same as in URL path)
reward_type	string	Y	Reward type
entry_fee	number	Y	Entry fee (eMGG)
min_players	number	Y	Minimum players
max_players	number	Y	Maximum players
entry_condition	number	Y	Minimum balance to enter (eMGG)

Request example

```
{
  "room_id": "room_beginner_100",
  "reward_type": "RANKING",
  "entry_fee": 1500,
  "min_players": 2,
  "max_players": 4,
  "entry_condition": 5000
}
```

Response (200 OK)

```
{
  "success": true,
  "room_id": "room_beginner_100"
}
```

Error responses

HTTP	Description
400	Bad Request
401	Unauthorized
404	Not Found (room_id not found)
500	Internal Server Error

7.2.3 Delete Room

Called when Admin deletes a room.

Item	Value
Method	`DELETE`
URL	`{main_url}/{game_rooms_url}/{room_id}`
Auth	`Authorization: Bearer {api_key}`
Body	None

Response (200 OK)

```
{
  "success": true,
  "room_id": "room_beginner_100"
}
```

Error responses

HTTP	Description
401	Unauthorized
404	Not Found
500	Internal Server Error

7.2.4 Room Status (Active/Inactive)

Called when Admin toggles room active/inactive.

Item	Value
Method	`PATCH`

Item	Value
URL	`{main_url}/{game_rooms_url}/{room_id}/status`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

Field	Type	Required	Description
is_active	boolean	Y	true: active, false: inactive

Request example

```
{
  "is_active": false
}
```

Response (200 OK)

```
{
  "success": true,
  "room_id": "room_beginner_100",
  "is_active": false
}
```

Error responses

HTTP	Description
400	Bad Request
401	Unauthorized
404	Not Found
500	Internal Server Error

7.3 Reward Rates API

7.3.1 Create Reward Rate

Called when Admin creates reward rates for a room and player count.

Item	Value
Method	`POST`

Item	Value
URL	`{main_url}/{game_reward_rates_url}`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

Field	Type	Required	Description
room_id	string	Y	Room identifier
player_count	number	Y	Number of players (e.g. 2, 4)
reward_rates	object	Y	Rank-based reward rate (%). Keys are rank strings "1","2",… ; values are numbers

Request example

```
{
  "room_id": "room_beginner_100",
  "player_count": 4,
  "reward_rates": {
    "1": 50,
    "2": 30,
    "3": 20
  }
}
```

Response (200 OK)

```
{
  "success": true,
  "room_id": "room_beginner_100",
  "player_count": 4
}
```

Error responses

HTTP	Description
400	Bad Request (e.g. rates sum > 100)
401	Unauthorized
409	Conflict (combination already exists)
500	Internal Server Error

7.3.2 Update Reward Rate

Called when Admin updates reward rates.

Item	Value
Method	`PUT`
URL	`{main_url}/{game_reward_rates_url}/{room_id}`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

Field	Type	Required	Description
room_id	string	Y	Room identifier
player_count	number	Y	Number of players
reward_rates	object	Y	Rank-based reward rates (%)

Request example

```
{
  "room_id": "room_beginner_100",
  "player_count": 4,
  "reward_rates": {
    "1": 55,
    "2": 28,
    "3": 17
  }
}
```

Response (200 OK)

```
{
  "success": true,
  "room_id": "room_beginner_100",
  "player_count": 4
}
```

Error responses

HTTP	Description
400	Bad Request

HTTP	Description
401	Unauthorized
404	Not Found
500	Internal Server Error

7.3.3 Delete Reward Rate

Called when Admin deletes reward rates for a room and player count.

Item	Value
Method	`DELETE`
URL	`{main_url}/{game_reward_rates_url}/{room_id}/{player_count}`
Auth	`Authorization: Bearer {api_key}`
Body	None

Response (200 OK)

```
{
  "success": true,
  "room_id": "room_beginner_100",
  "player_count": 4
}
```

Error responses

HTTP	Description
401	Unauthorized
404	Not Found
500	Internal Server Error

7.4 Items API

7.4.1 Create Item

Called when Admin creates an item.

Item	Value
Method	`POST`
URL	`{main_url}/{game_items_url}`

Item	Value
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

Field	Type	Required	Description
item_id	string	Y	Item identifier (value created in MGG Arena)
item_name	string	Y	Item name
item_description	string	Y	Item description
item_price	number	Y	Price (eMGG)
item_type	string	Y	Item type (e.g. `character`, or game-defined values)

Request example

```
{
  "item_id": "item_skin_001",
  "item_name": "Rare Skin",
  "item_description": "Limited edition character skin",
  "item_price": 500,
  "item_type": "character"
}
```

Response (200 OK)

```
{
  "success": true,
  "item_id": "item_skin_001"
}
```

Error responses

HTTP	Description
400	Bad Request
401	Unauthorized
409	Conflict (item_id already exists)
500	Internal Server Error

7.4.2 Update Item

Called when Admin updates item information.

Item	Value
Method	`PUT`
URL	`{main_url}/{game_items_url}/{item_id}`
Auth	`Authorization: Bearer {api_key}`
Content-Type	`application/json`

Request body

Field	Type	Required	Description
item_id	string	Y	Item identifier (same as in URL path)
item_name	string	Y	Item name
item_description	string	Y	Item description
item_price	number	Y	Price (eMGG)
item_type	string	Y	Item type

Request example

```
{
  "item_id": "item_skin_001",
  "item_name": "Rare Skin (Updated)",
  "item_description": "Limited edition character skin",
  "item_price": 600,
  "item_type": "character"
}
```

Response (200 OK)

```
{
  "success": true,
  "item_id": "item_skin_001"
}
```

Error responses

HTTP	Description
400	Bad Request
401	Unauthorized

HTTP	Description
404	Not Found
500	Internal Server Error

7.4.3 Delete Item

Called when Admin deletes an item.

Item	Value
Method	`DELETE`
URL	`{main_url}/{game_items_url}/{item_id}`
Auth	`Authorization: Bearer {api_key}`
Body	None

Response (200 OK)

```
{
  "success": true,
  "item_id": "item_skin_001"
}
```

Error responses

HTTP	Description
401	Unauthorized
404	Not Found
500	Internal Server Error

7.5 HTTP Status Codes and Error Body

Code	Meaning
200	Success (see examples above for body)
201	Created (optional for create operations)
400	Bad Request (missing required fields, format error, etc.)
401	Unauthorized (API Key missing/mismatch)
404	Not Found (resource not found)
409	Conflict (e.g. duplicate create)

Code	Meaning
500	Internal Server Error

Error body example (recommended):

```
{
  "success": false,
  "error": "Error message",
  "room_id": "room_001",
  "item_id": "item_001"
}
```

7.6 Notes

- Admin **does not parse** the response body; it only uses the HTTP status to determine success or failure. Return 2xx with JSON on success.
- The **API Key** is the same key issued when the game was registered with MGG Arena. Admin sends it as `Authorization: Bearer {api_key}`; the game server must validate it before processing.
- Room/reward/item data is **saved to the MGG Arena DB first** by Admin, then the game server API is called. If the game server API fails, the MGG Arena DB is already updated; the game server may implement its own sync or correction if needed.

---

8. Document Information

- **Document title**: MGG Arena Game Integration Guide (Full Version) — MGGARENA_Game_Guide_Full
- **Version**: 1.0
- **Last updated**: 2026-01-30

This document is a full-version integration of GAME_API_REFERENCE, GAME_SERVER_DEVELOPMENT_GUIDE, and GAME_SERVER_API_REFERENCE. For a summary version, see MGGARENA_Game_Guide.en.md.